

VELDORA STUDIO

Development Proposal

Weather Monitoring and Alerting System — Sydney Metro M1 Line

In response to the Technical Proposal for Weather Monitoring, Sydney Metro M1 (OBS-MTS-M1-WX-2026-01, Rev A, 22 July 2026)

Submitted by	Veldora Studio
Contact	Hassan Ali · hassanali@veldorastudio.com
Prepared for	Observator Instruments (Observator Group)
Scope	Ingestion pipeline, central server, web portal and mobile application
Duration	10 months from kickoff
Proposal reference	VS-OBS-MTS-M1-2026-01
Version / date	Rev A · September 2026
Status	Commercial-in-confidence

Contents

- 1. What we are building 3
- 2. The seven stations 3
- 3. Data rate — and why it drives the architecture 3
- 4. Ingestion — the part that must never lose data 4
- 5. Security 5
- 6. Performance — millions of records without slowing down 6
- 7. The alert engine..... 6
- 8. The web portal..... 7
- 9. The mobile application 8
- 10. Month-by-month plan 9
- 11. Requirement traceability..... 11
- 12. What we need from you..... 12
- 13. Assumptions 12
- 14. Ways of working 13
- 15. Acceptance criteria 13
- 16. In short..... 13

1. What we are building

A complete monitoring and alerting platform for seven trackside weather stations on the Sydney Metro M1 line, covering rainfall, flood water level, temperature, humidity and wind — plus supervised control of the trackside flood pumps at Marrickville.

This is a **safety-critical rail system**. Its alerts trigger front-of-train patrols, temporary speed restrictions, and blocking of the line. The engineering standard throughout is that **no reading is ever lost and no alert is ever missed**, and that every design decision can be explained in those terms.

The four deliverables

#	Deliverable	Technology
1	Ingestion pipeline — from the trackside logger to the database, losing nothing	Secure agent on AWS Lightsail → AWS SQS → NestJS workers
2	Central server — storage, alert engine, API	NestJS (TypeScript), MongoDB, Redis
3	Web portal — map, trends, station and pump screens	Next.js (TypeScript), responsive
4	Mobile application — native iOS and Android	Flutter

2. The seven stations

Each station carries only the sensors its location requires.

#	Location	Rain	Water level	Float switch	Temp / Humidity	Wind	Pumps
1	Marrickville	✓	✓	✓	—	—	✓
2	Marrickville–Dulwich Hill	—	✓	✓	✓	✓	—
3	Canterbury	—	✓	✓	—	—	—
4	Campsie	—	✓	✓	—	—	—
5	Belmore triangle	✓	✓	✓	✓	✓	—
6	Lady Game Drive (tunnel)	—	✓×2	✓×2	—	—	—
7	Windsor Road SSC	—	—	—	✓	✓	—

Location 6 is two independent units — up tunnel and down tunnel — each with its own logger. The platform therefore handles **eight loggers across seven locations**, presenting Location 6 as one place with two monitoring points.

3. Data rate — and why it drives the architecture

The datalogger will send **either every second or every minute**, and for pump operation **one second is critical**. We design for one second throughout.

Loggers	8
Readings per second	8
Per day	691,200
Per month	~21 million

Per year	~250 million
----------	--------------

A quarter of a billion readings a year is entirely manageable — but only with a storage and query design chosen for it from day one, not retrofitted. Section 6 sets out exactly how.

How much of it we keep is a decision to make together

That yearly figure assumes every 1-second reading is kept indefinitely, which is almost certainly more than is needed and directly drives the hosting cost.

We will agree the retention policy with you in **Month 1**, before any of it is built, because it shapes the storage design. The realistic options:

Option	1-second detail kept	Aggregates kept	Roughly
Lean	1 month	forever	smallest and cheapest
Balanced	3–6 months	forever	our usual recommendation
Full	12 months+	forever	largest, for detailed incident review

In every option the **per-minute, per-hour and per-day aggregates are kept permanently**, so trends, daily summaries, reporting and year-on-year comparison are unaffected by the choice. What varies is only how far back you can replay second-by-second detail — which matters chiefly for investigating a specific incident.

Our recommendation is to keep full detail for a window comfortably longer than any likely investigation, and aggregates forever. If a shorter window is preferred for cost, the system is designed so the setting can be changed later without redesign or data migration.

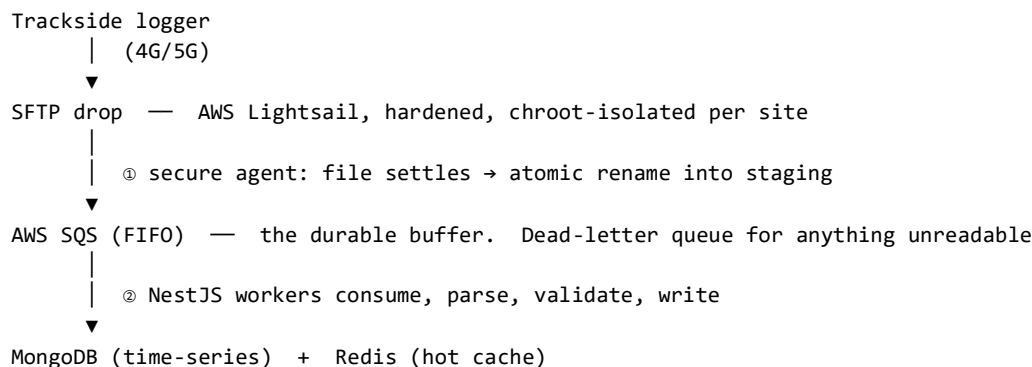
Where real-time control actually lives

The pump start/stop decision runs **on the OMC-048 logger at the station**, in real time, independent of the network. That is correct and we will not move it: a pump that stops working because a cellular link dropped is not acceptable on a flood site.

The central server's job is to **aggregate, alert, display, record and supervise** — and to accept a manual override from an authorised operator. We will hold the server to the Statement of Requirements' obligation: **weather alerts within 5 minutes, system-fault alerts within 30 minutes**, measured end to end.

4. Ingestion — the part that must never lose data

Every reading matters, so the pipeline is built so that a failure anywhere **delays** data rather than **losing** it.



↓
Alert engine → screen · email · SMS

Why SQS sits in the middle

Without a queue, a database hiccup or a deployment means readings arriving in that window are gone. With SQS:

- **Nothing is lost on failure.** A message stays on the queue until a worker confirms it was written. A crashed worker means the message reappears, not that it vanishes.
- **Bursts are absorbed.** A logger reconnecting after an outage can deliver hours of buffered readings at once without overwhelming the database.
- **Poison data cannot block the queue.** Anything unparseable goes to a dead-letter queue and is reported, while everything behind it keeps flowing.
- **Workers scale independently.** More load means more consumers, with no change to the ingest path.
- **FIFO where order matters,** so a station's readings are processed in the order they were taken.

Guarantees we build in

Guarantee	How
No duplicates	Every file carries a content hash; a repeat is recognised and skipped
No loss on crash	Rename-into-staging before sending; the file is only archived once the server confirms
No loss on outage	Files accumulate on disk and are replayed automatically on reconnection
Nothing deleted	Ingested files are archived permanently, never removed
Out-of-order safe	Readings are keyed by their own timestamp, not arrival order
Auditable	Every file's journey is recorded — received, parsed, stored, archived

5. Security

Data security is treated as a requirement of the same rank as correctness.

In transit

- **SFTP only** from the trackside box, on hardened OpenSSH with key-based authentication and per-site chroot isolation, so one site can never see another's data.
- **TLS everywhere else** — logger to server, server to database, browser and mobile to API.
- The ingest agent holds an **outbound-only credential**. The trackside box exposes **no inbound port** to us.

At rest and in the platform

- **Encryption at rest** on the database and on backups.
- **Least-privilege credentials** — the ingest credential can upload readings and nothing else. It cannot read other customers' data, change configuration, or administer anything.
- **Every secret hashed**, never stored recoverably. Generated passwords are shown once.
- **Role-based access control** with per-station scoping, so a user sees only what their role and their sites permit.
- **Optional multi-factor authentication** for portal access.
- **Full audit trail** — every login, threshold change, acknowledgement and manual pump command recorded against a named person with a timestamp.

- **Automated dependency scanning** in the build, and a **penetration test** before go-live.

Scripts on the Lightsail box

Everything privileged is a single root-owned script with a single narrow sudo rule. Arguments are validated in three independent layers, so no one check is load-bearing. The agent runs unprivileged, cannot modify its own code, and can execute exactly one command as root — nothing else.

6. Performance — millions of records without slowing down

The requirement is that a year of second-by-second history stays fast to query and fast to display. That is a design problem, and these are the decisions.

Storage

- **Time-series collections**, purpose-built for this shape of data — high write rate, append-only, queried by time range.
- **Compound indexes** matched to the queries that actually run, verified with query plans rather than assumed.
- **Pre-aggregated rollups**. Every reading is summarised into per-minute, per-hour and per-day records as it arrives. A chart of last month reads ~720 hourly rows, not 21 million raw ones.
- **Tiered retention** — full 1-second detail for the recent window, aggregates kept for the long term. Nothing needed for reporting is ever discarded.

Serving

- **Redis** for live readings and dashboard state, so the map and station tiles do not touch the database on every refresh.
- **Cursor pagination** everywhere, so a large result set never loads at once.
- **Streamed CSV export**, so a 90-day export cannot exhaust server memory.
- **WebSocket push** for live values — the browser is told when something changes rather than polling for it.

In the browser and on the phone

- Server-side rendering and code splitting so the first screen is fast on a field connection.
- Charts render **downsampled** series matched to the pixels available; the underlying detail is fetched only when zoomed.
- Virtualised tables, so a 50,000-row query scrolls smoothly.
- A performance budget enforced automatically in the build, so a regression fails the pipeline rather than reaching production.

7. The alert engine

The heart of the system, and the part with the least tolerance for error.

Rainfall — vigilance logic

For each threshold — **≥25 mm/hr**, **≥45 mm/3 h**, **≥120 mm/3 days** — the engine maintains a live rolling total and compares it continuously.

When a threshold is crossed it raises the alert immediately with the exact MTS wording. When rainfall falls back below the line it does **not** stand down: it starts a **vigilance countdown** — **6 h / 12 h / 48 h** respectively — and

holds the alert for the whole period. Further qualifying rain **restarts** the countdown. When the clock finally expires it issues an automatic **all-clear**.

Every value — thresholds, windows, countdown durations, wording — is a configuration item MTS can change without a software release.

Flood, temperature and wind

Parameter	Logic
Flood	Standing water (PTZ verification prompt) → water above rail foot (block the line) → trending down (staged reinstatement 25 → 60 kph → unrestricted)
Temperature	≥38 °C and ≥45 °C rising, sustained >5 min; falling <45 °C / <38 °C sustained >10 min; drives the 60/40 kph TSR and heat-patrol wording
Wind	≥75 / ≥85 kph TSR wording at the listed kilometrages; ≥120 kph block-line on all external track
Pumps	Start, stop, fail, no-flow, high-high level, sensor discrepancy — all raised immediately
System	Sensor offline, missing data, low battery, communications loss — leading indicators, not just failures

Redundancy and fallback

On sensor failure the engine automatically generates alerts from the **designated alternate source** and **rewords** the alert to state which additional sections it now covers. A fault alert is issued first, telling MTS that alerting has moved to the fallback — so nobody is unaware that they are running degraded.

Delivery

Screen, email and SMS. Each alert carries date, time, **track (up/down)**, **rail (up/down)**, **kilometrage**, and a **direct hyperlink to the event**. For standing-water conditions it carries a **one-click PTZ camera link** — on screen, in the email, and as a short URL in the SMS.

SMS goes through a provider with an Australian delivery guarantee, with a second provider configured as failover and delivery receipts recorded, so the 5-minute obligation can be **evidenced**, not just claimed.

8. The web portal

One responsive application — phone, tablet, laptop — with no separate mobile site to maintain.

Screen	Contents
Corridor map	Every station on a map, live readings shown against their thresholds, colour-coded status, system-health summary and a station-status table. Click a site for its detail — the top-level view MTS asked for
Station detail	All sensors for that location, live and historical, with thresholds drawn on the charts
Trends	Time-series for wind (mean and gust), rainfall intensity, temperature, humidity and level, with threshold lines and flexible date ranges
Flood event	Water-level series for adjacent locations, pump auto-start markers, pump status, affected-vicinity inset, timestamped event log
Marrickville pump station	Live sensors, level trend against pump-start and high-high lines, duty/standby status and condition, AUTO/MANUAL toggle , supervised START/STOP with confirmation

System health	Refreshed at ≤10 minutes: inoperable, unresponsive, missing sensor or pump data, low battery and other leading indicators
Alerts & notifications	Full searchable history, filter by severity / acknowledgement / location / date, unacknowledged badge, per-event acknowledge, CSV export
Historical query	Point-and-click by location, sensor and date range with optional averaging — no technical knowledge, no SQL. Paged sortable results, one-click CSV export
Administration	Users, roles, station scoping, thresholds, alert wording, recipients

Users and roles

Twenty or more named logins, each individually authenticated. Six standard roles, tailorable, with a user able to hold more than one:

Administrator · Operator · Pump Controller · Maintainer/Technician · Analyst/Reporting · Viewer

Each user's access is scoped to specific stations or areas. MTS administers all of this themselves — adding, editing, suspending and removing users — with no dependency on us for day-to-day account changes.

9. The mobile application

A native application for **iOS and Android**, built in **Flutter** from one codebase.

We propose a **native app rather than a web wrapper**. A wrapper cannot deliver push notifications reliably, cannot alert when the app is closed, and cannot work offline — and on a safety-critical alerting system those are the properties that matter most. The cost difference is modest; the capability difference is not.

Capability	
Push notifications	Alerts arrive on the phone within seconds, even with the app closed or the phone locked
Map and station views	The same corridor map and per-site detail, laid out for a phone
Live readings and trends	Touch-friendly charts
Acknowledge alerts	From the phone, recorded against the named user
Supervised pump control	For authorised roles only, with confirmation — so on-call staff can act without returning to a desk
Offline tolerance	Last-known readings remain visible on a poor connection; actions queue and send on reconnect
Biometric unlock	Face/fingerprint, with MFA where enabled

10. Month-by-month plan

Ten months from kickoff. Each month ends with something demonstrable, and the first working end-to-end path exists in **Month 2** rather than at the end.

Month 1 — Architecture, decisions, and secure ingest

Month 1 is deliberately weighted toward **asking questions and writing the answers down**. Most of what goes wrong in a system like this is decided in the first fortnight and discovered in the last — a retention rule chosen carelessly, an alert path nobody agreed on, a data format assumed rather than seen.

So we start with a structured architecture review: we bring the open questions, work through them with your team, and record each decision with its reasoning and its consequences. That record becomes the reference for the following nine months, and it is what lets us build once rather than twice.

Topics we will close out together:

- **Data:** exact logger output format, transmission method, transmit interval, and what a "complete" message looks like.
- **Retention:** how long 1-second detail is kept, what is aggregated, and what must survive to contract end (see §3).
- **Alerting:** every threshold, dwell timer, vigilance period and the exact wording — including the values currently marked "pending confirmation".
- **Failure behaviour:** what the system should do when a sensor, a link, the database or the cloud region is unavailable, and what it must never do.
- **Security and residency:** AWS region, data residency, any MTS cyber standard to be assessed against, and MFA policy.
- **Roles:** who may see what, who may acknowledge, and who may operate a pump.
- **Integration:** PTZ camera access, SMS provider, and the pump panel interface.

Week	Deliverable
1	Kickoff; architecture workshop — open questions raised and answered ; decisions recorded with rationale; AWS accounts, environments, CI/CD; security baseline
2	Lightsail SFTP box hardened: per-site chroot, key-only auth, no inbound port for the agent
3	Ingest agent: stability gates, atomic staging, retry with backoff, permanent archive
4	SQS queues and dead-letter queues provisioned; agent publishing; end-to-end message flow proven

Month 2 — Ingestion at 1 Hz, storage, and no data loss

Week	Deliverable
1	NestJS workers consuming SQS; parser for the logger's format; validation and plausibility checks
2	Time-series schema, indexes and retention; write path benchmarked at full 1-second rate
3	Idempotency (content hashing), out-of-order handling, replay of a backlog after an outage
4	Failure drills: kill a worker mid-write, take the database offline, disconnect the box — prove nothing is lost. First readings visible end to end

Month 3 — All sensors, provenance and backfill

Week	Deliverable
1	Rain, level, float, temperature, humidity, wind, battery/solar telemetry all parsed and stored
2	Per-station sensor configuration; unit handling; calibration offsets
3	Rolling aggregates (minute/hour/day) computed as data arrives
4	Historical backfill tooling; data-quality reporting; gap detection

Month 4 — Alert engine core

Week	Deliverable
1	Configurable threshold table — every value a setting, not code
2	Wind and temperature logic including dwell timers and rising/falling hysteresis
3	System-fault detection: sensor offline, missing data, low battery, comms loss
4	Alert lifecycle — raise, acknowledge, clear, audit — with full history

Month 5 — Rainfall vigilance and flood logic

Week	Deliverable
1	Rolling 1 h / 3 h / 3-day accumulation, continuously maintained
2	Vigilance timers (6/12/48 h) with restart-on-new-rain and automatic all-clear, surviving restarts
3	Flood state machine: standing water → above rail foot → trending down, with staged reinstatement
4	Redundancy fallback with automatic alert re-wording, and the fault alert that announces it

Month 6 — Alert delivery

Week	Deliverable
1	On-screen alerting over WebSocket, with pop-ups and the unacknowledged badge
2	Email channel with full alert content and deep links
3	SMS with primary and failover providers, delivery receipts, and evidence of the 5-minute obligation
4	PTZ camera links on screen, in email and by SMS; end-to-end timing measured and reported

Month 7 — Portal: map, trends, history

Week	Deliverable
1	Application shell, authentication, RBAC, station scoping, MFA
2	Corridor map with live status, click-through to station detail
3	Trends and station screens with thresholds drawn; chart performance work
4	Historical query engine, paged results, one-click CSV export

Month 8 — Portal: operations

Week	Deliverable
------	-------------

1	Marrickville pump screen — live sensors, level trend, pump status and condition
2	AUTO/MANUAL toggle and supervised START/STOP , confirmation, role-gated, fully audited
3	Flood-event screen with pump markers and the event log
4	System-health dashboard (≤ 10 -minute refresh); administration screens for users, roles, thresholds and recipients

Month 9 — Mobile application

Week	Deliverable
1	Flutter foundation, authentication, biometric unlock, secure token storage
2	Map, station detail, live readings and trends on the phone
3	Push notifications , alert acknowledgement, offline tolerance
4	Supervised pump control for authorised roles; TestFlight and Play Console beta builds

Month 10 — Hardening, performance and handover

Week	Deliverable
1	Load test at full rate — sustained 1 Hz across all eight loggers, plus burst replay of a multi-day backlog
2	Security review and penetration test ; dependency audit; remediation
3	High availability, backup and restore rehearsal (a backup is not a backup until it has been restored); failover drill
4	Documentation, runbooks, operator and maintainer training, SAT support, handover

11. Requirement traceability

Every item in the Statement of Requirements, mapped to the month that delivers it. Nothing is left unassigned.

Requirement	Delivered in
Rainfall measurement, accumulation windows, thresholds	M3, M5
Rainfall vigilance timers (6/12/48 h) and all-clear	M5
Flood level, float switch verification, staged reinstatement	M3, M5
Temperature thresholds with dwell timers	M4
Wind thresholds and TSR/block-line wording	M4
Pump status, alarms and leading indicators	M4, M8
Supervised manual pump control	M8
Configurable threshold and wording table	M4
Redundancy and fallback with re-wording	M5
Screen / email / SMS within 5 minutes	M6
System-fault alerts within 30 minutes	M4, M6
Alert content: date, time, track, rail, kilometrage, hyperlink	M6
PTZ camera verification links	M6
Alert and notification history, acknowledgement, export	M4, M7
Corridor map with click-through detail	M7
Trends and per-station screens	M7

Flood-event screen	M8
System health, ≤10-minute refresh	M8
Historical query engine and CSV export	M7
20+ named logins, RBAC, six roles, station scoping	M7
MFA	M7
Audit trail	M4, M7
Responsive HMI, phone to desktop	M7
Mobile application with push	M9
Solar/battery telemetry displayed and alarmed	M3, M8
Data transmission security	M1
No data loss / queueing	M1, M2
Availability and planned-maintenance obligations	M10
Performance at scale	M2, M6, M10
Backup, restore and failover	M10
Documentation, runbooks, training	M10

12. What we need from you

Development can start without these, but each becomes blocking at the month shown.

#	Needed	By
1	The logger's output format — a real sample file per sensor type, and confirmation of the transmission method	M1
2	Confirmed threshold values and exact alert wording currently marked "pending confirmation"	M4
3	Kilometrage, track and rail designations per station, and GPS positions	M6
4	PTZ camera URLs plus any authentication and network access needed to reach them	M6
5	SMS provider decision — or authority for us to select one — and the recipient list	M6
6	AWS region and any data-residency requirement	M1
7	Any MTS cyber-security standard the system must be assessed against	M1
8	The 20 portal users, their roles, and their station scope	M7
9	Apple Developer and Google Play accounts for store publication	M9
10	Retention decision — how long 1-second detail is kept (see §3) — and the data-handover format at contract end	M1

13. Assumptions

- The logger performs its own real-time pump control locally; the server supervises, records and alerts, and offers manual override.
- Sensors, loggers, masts, power, installation, EMC compliance and calibration are supplied and certified by Observator. Our scope is the software platform.
- Cellular connectivity to each station is provided and maintained by Observator.
- The trackside pump control panel accepts a remote start/stop demand and provides run and fault status.
- Cloud infrastructure is billed to Observator or MTS directly; our estimates exclude hosting fees.

14. Ways of working

- **Two-week sprints** with a working demonstration at the end of each.
- **Fortnightly progress reports** and a monthly review.
- **Automated tests** on every change: unit, integration and end-to-end.
- **Every alert path tested against real recorded data**, not synthetic input.
- **Staging environment** mirroring production, for MTS to review before release.
- **Code review on every change**, with security-sensitive changes reviewed twice.
- **Documentation written as we go**, not assembled at the end.

15. Acceptance criteria

The system is complete when, demonstrably:

1. Every reading from every logger reaches the database, proven by a failure drill that kills workers and severs the network mid-flight.
2. A threshold breach produces screen, email and SMS alerts **within 5 minutes**, evidenced by delivery receipts.
3. Rainfall vigilance holds, restarts and clears correctly across a simulated multi-day event.
4. A sensor failure moves alerting to its fallback source and re-words the alert automatically.
5. An authorised operator can start and stop the Marrickville pumps from the portal and the phone, with the action recorded against their name.
6. A year of 1-second data can be queried and charted without degradation.
7. Twenty users can work concurrently with role and station scoping enforced.
8. A penetration test is passed with no unresolved high-severity findings.
9. A backup has been **restored** and verified, not merely taken.

16. In short

- **10 months**, four deliverables, nothing in the Statement of Requirements left unassigned.
- **Month 1 is questions and answers**, not code — every architectural decision taken with you and written down, including how long data is kept.
- **AWS SQS** at the centre of ingestion, so a failure delays data rather than losing it.
- Designed for **1-second data from day one** — 21 million readings a month, and fast to query at a year's scale.
- **NestJS + Next.js + Flutter**, with security and performance treated as requirements rather than afterthoughts.
- A **native mobile app**, because push notifications are the point of a safety-critical alerting system and a web wrapper cannot deliver them.